BIGQUERY THEORY Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wa

1. What is Google BigQuery?

BigQuery is Google's fully managed, serverless data warehouse that is mainly used for analytics on very large datasets. It is built for running SQL queries at scale, across terabytes or even petabytes of data, without me worrying about infrastructure. The main advantage is that it separates storage and compute, so I can store a lot of data cheaply and only pay for compute when I query. It uses columnar storage, which makes aggregation queries fast, and everything runs in parallel under the hood. From my perspective as a data engineer, I see it as a tool that gives me simple SQL access to very large data, while Google manages all scaling, patching, and performance tuning in the background.

2. How does BigQuery differ from traditional (transactional) databases?

BigQuery is designed for analytics, not for transactions. Traditional databases like MySQL or PostgreSQL are optimized for OLTP workloads where many small updates, inserts, and deletes happen frequently. They store data row by row because row-based storage is good for reading and writing complete records quickly. BigQuery, on the other hand, is an OLAP system. It uses columnar storage so that queries scanning specific columns across billions of rows are very fast. Instead of managing servers, indexes, and partitions myself, BigQuery handles those automatically. In short: transactional databases are good for real-time applications like banking or e-commerce transactions, while BigQuery is good for reporting, dashboards, machine learning, and large-scale data analysis.

3. Describe the architecture of BigQuery (high level: Dremel, Colossus, Borg, etc.).

At a high level, BigQuery runs on top of Google's internal infrastructure:

- Colossus is Google's distributed storage system where BigQuery stores data. It provides columnar storage, replication, and compression, which makes data durable and efficient.
- Dremel is the execution engine that actually processes SQL queries. It breaks queries into trees of smaller tasks and runs them in parallel across thousands of nodes. This is why queries return results in seconds, even for petabytes of data.
- Borg is the cluster manager that allocates resources for running those queries. It handles scheduling and fault tolerance so that I don't have to manage any servers.
- Jupiter is the high-speed network that connects compute and storage, making sure the query engine can fetch data very quickly.

So, the architecture is basically: Colossus for storage, Dremel for query execution, Borg for resource management, and Jupiter for fast data movement. Together they make BigQuery scalable and serverless.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

4. What is a dataset in BigQuery?

A dataset in BigQuery is like a container or namespace inside a project. It is the second-level hierarchy under the project. Inside a dataset, I can have tables, views, and materialized views. Using datasets helps me organize data logically, for example, I might create a dataset called sales_data to keep all tables related to sales. Datasets also allow me to set permissions at the dataset level, so I can control who can access which group of tables. This is important in large organizations where different teams work on different data domains.

5. How do you load data into BigQuery? Describe the different methods (batch load, streaming, connectors, Dataflow, Data Transfer Service, federated sources, etc.).

There are multiple ways to bring data into BigQuery, depending on the use case:

- **Batch Load**: I can upload files like CSV, JSON, Avro, Parquet, ORC, or Arrow from Cloud Storage. This is good for large historical data loads.
- **Streaming Inserts**: I can send real-time data directly into BigQuery using the streaming API. This is useful for event data like logs, clicks, or IoT signals where latency matters.
- **Connectors**: BigQuery has native connectors with tools like Dataflow, Dataproc, and also external tools like Apache Beam. For example, I can create ETL pipelines in Dataflow that load processed data into BigQuery.
- **Data Transfer Service**: This is a managed service for scheduled imports from external SaaS applications (like Google Ads, YouTube, or Salesforce) and from Google Cloud Storage or other BigQuery projects.
- **Federated Queries**: Instead of moving data, BigQuery allows me to query data in external sources directly, like Cloud Storage files, Google Sheets, or Cloud Bigtable.

So overall, I can choose batch loading for big historical loads, streaming for real-time, transfer service for automated SaaS imports, or federated queries when I don't want to duplicate the data.

6. Why is Google Cloud Storage (GCS) commonly used as a staging/secondary storage layer when loading data into BigQuery?

GCS is often used as a staging layer because it is cheap, durable, and can handle almost unlimited data. When I have raw data from different sources, I usually first land it into GCS in its native format like CSV, JSON, Avro, or Parquet. From there, I can clean and transform it, and then load it into BigQuery in a structured way. This staging step gives flexibility for example, if something fails during the load, I don't lose the data since it's safely stored in GCS. It also allows me to reprocess or reload data without going back to the original source system. Another reason is that batch loading into BigQuery directly from GCS is highly optimized and cheaper compared to streaming. So, GCS works as a durable buffer between source systems and BigQuery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. How do streaming inserts work in BigQuery and when should you use them?

Streaming inserts let me send individual rows of data directly into BigQuery using the streaming API. Instead of waiting to batch files and load them, I can push events in real time and they become available for querying almost immediately (usually within a few seconds). This is very useful when I need low-latency analytics, like monitoring dashboards, fraud detection, or IoT sensor data.

However, streaming has some trade-offs: it costs more compared to batch loads, and streaming data cannot be updated or deleted until it's written to the storage layer. It's also not covered by the free tier like batch loads. So, I use streaming only when real-time access is critical. Otherwise, I prefer to load data in batches through GCS for cost efficiency.

8. What is the BigQuery Data Transfer Service and what is it used for?

The BigQuery Data Transfer Service (DTS) is a managed service that automates moving data from external sources into BigQuery on a scheduled basis. It's especially useful for pulling data from SaaS applications or Google services without building custom ETL pipelines. For example, I can automatically import data from Google Ads, YouTube Analytics, Google Cloud Storage, or even external apps like Salesforce. I just set up the connection and schedule, and BigQuery keeps pulling fresh data for me.

I use it when I want a no-code or low-code solution to bring in third-party or Google service data into BigQuery consistently, without worrying about pipeline maintenance.

9. What are partitioned tables in BigQuery and why are they important?

Partitioned tables in BigQuery are tables that are divided into smaller, manageable parts based on a column or ingestion time. The most common type is date-based partitioning, where each partition stores data for a specific day. This is important because when I query the table, BigQuery only scans the partitions that are needed, instead of scanning the entire dataset. This reduces query costs and improves performance.

For example, if I have a table with 5 years of log data but I only need last week's data, partitioning ensures BigQuery only reads those partitions. It also helps with data lifecycle management because I can set automatic expiration on older partitions to save storage costs.

10. What is clustering in BigQuery and how does it differ from partitioning?

Clustering in BigQuery organizes the data inside a table based on the values of one or more columns, like customer_id or region. It doesn't split the table into separate physical partitions, but it arranges data so that rows with similar values are stored close together. This makes queries faster when filtering or aggregating on those clustered columns.

The difference from partitioning is:

- Partitioning reduces data scanned by splitting the table into large chunks (for example, by date).
- Clustering reduces data scanned within each partition by grouping related rows together, which makes filtering more efficient.

In practice, I often use them together: partition by date to filter on time ranges, and cluster by columns like user_id or country to improve performance within those partitions.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. What are BigQuery slots and how do they affect query performance?

BigQuery slots are the units of computational capacity that BigQuery uses to run SQL queries. Think of them as virtual CPUs inside Google's infrastructure. When I run a query, BigQuery breaks it into smaller tasks and distributes them across many slots. The more slots available, the faster the query can finish because more parallel processing is happening.

By default, in on-demand pricing, BigQuery automatically manages slots for me. But in flat-rate pricing, organizations can purchase dedicated slots and allocate them to different projects or teams. In that case, if there aren't enough slots, queries might queue and wait until resources free up.

So, query performance depends on how many slots are working on my query and how efficiently my SQL uses them. Partitioning, clustering, and reducing scanned data help queries run faster and need fewer slots, which improves overall performance.

12. How does BigQuery achieve high query performance and what techniques do you use to optimize queries?

BigQuery achieves high performance mainly through its architecture:

- It stores data in a columnar format, so aggregations and scans across columns are fast.
- It separates compute and storage, and the Dremel engine executes queries in parallel across thousands of slots.
- It compresses and distributes data in Colossus, which means queries can read less data and spread it across multiple nodes.

As a data engineer, I use several techniques to optimize queries:

- Partition tables by date or another logical field so queries only scan the relevant data.
- Cluster tables on frequently filtered columns to make scans more efficient.
- Select only the columns I need instead of using SELECT *.
- Use materialized views for repeated heavy aggregations.
- Use approximate functions like APPROX_COUNT_DISTINCT when exact precision isn't needed.
- Avoid repeatedly joining massive tables unnecessarily sometimes denormalizing data works better in BigQuery.
- Monitor query execution using the query plan to identify stages that consume the most resources.

These techniques make queries cheaper and faster while still taking advantage of BigQuery's distributed power.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

13. What are the best practices for managing and monitoring BigQuery costs?

Cost management in BigQuery comes down to controlling how much data gets stored and how much data gets scanned. The best practices I follow are:

- Use partitioning and clustering so queries scan only the required data.
- **Avoid SELECT *** and instead query only the needed columns.
- Leverage table expiration policies to automatically delete old staging or temporary data.
- Use materialized views for recurring heavy queries, so I don't reprocess large datasets every time.
- Set budgets and alerts in GCP so I get notified before costs get too high.
- Review the BigQuery audit logs and INFORMATION_SCHEMA tables to see which queries are consuming the most resources.
- Use query dry-run mode to check how many bytes a query will scan before actually running it.
- Consider reservations (flat-rate slots) if the workload is predictable and high-volume, since that can be cheaper than on-demand pricing.

By combining technical optimization with monitoring and governance, I can keep BigQuery costs under control while still delivering fast analytics.

14. What data types does BigQuery support?

BigQuery supports a wide range of data types so that I can model different kinds of data properly. The main categories are:

- Numeric types: INT64 (integer), FLOAT64 (floating-point), NUMERIC (fixed precision up to 38 digits), and BIGNUMERIC (even higher precision).
- Boolean: TRUE/FALSE values.
- String types: STRING (UTF-8 encoded text) and BYTES (binary data).
- Date and time types: DATE, DATETIME, TIME, and TIMESTAMP. These handle different levels of granularity and time zones.
- Complex types: ARRAY (a list of values of the same type) and STRUCT (similar to a row or object with multiple fields).
- Geography: GEOGRAPHY type for spatial data like points, lines, polygons.
- JSON: JSON type for storing semi-structured JSON documents.

These data types give flexibility for example, I can store semi-structured data in JSON or arrays, but still query it with SQL.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

15. How do JOIN operations work in BigQuery? (include INNER, LEFT, RIGHT, FULL, CROSS - and examples)

JOIN operations in BigQuery work similar to SQL joins in other databases, but because BigQuery is designed for massive datasets, I need to be mindful of performance (like making sure join keys are not skewed).

- **INNER JOIN**: Returns rows where keys match in both tables.
  Example: Get orders and their matching customers.

```
SELECT o.order_id, c.customer_name

FROM orders o

INNER JOIN customers c

ON o.customer_id = c.customer_id;
```

- **LEFT JOIN**: Returns all rows from the left table, even if there's no match in the right.
  Example: Get all customers, even those without orders.

```
SELECT c.customer_name, o.order_id

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id;
```

- **RIGHT JOIN**: Opposite of LEFT JOIN returns all rows from the right table, with NULLs if no match in the left.

```
SELECT c.customer_name, o.order_id

FROM customers c

RIGHT JOIN orders o

ON c.customer_id = o.customer_id;
```

- **FULL JOIN**: Returns all rows from both tables, with NULLs when there's no match.

```
SELECT c.customer_name, o.order_id

FROM customers c

FULL JOIN orders o

ON c.customer_id = o.customer_id;
```

- **CROSS JOIN**: Returns the Cartesian product (every row from one table with every row from the other). Used rarely, like for generating combinations.

```
SELECT a.product, b.region

FROM products a

CROSS JOIN regions b;
```

In BigQuery, JOINs are powerful but can be expensive, so I try to reduce data before joining by using partition filters or pre-aggregating.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

16. What is the QUALIFY clause in BigQuery and how is it used?

The QUALIFY clause lets me filter the results of window (analytic) functions directly, without wrapping the query in another subquery. Normally, window functions like ROW_NUMBER or RANK are calculated after the WHERE clause, so I can't filter them directly. QUALIFY solves this.

Example: Suppose I want the latest order per customer.

```
SELECT customer_id, order_id, order_date,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) as rn
FROM orders
QUALIFY rn = 1;
```

Here, QUALIFY filters the result of the ROW_NUMBER function so I only get the latest order for each customer. Without QUALIFY, I'd need to wrap this query in another SELECT. It makes queries cleaner and easier to read.

17. Explain window (analytic) functions in BigQuery and give an example of use.

Window (or analytic) functions in BigQuery allow me to perform calculations across a set of rows that are related to the current row, without collapsing them into a single result like GROUP BY would. They are defined using the OVER() clause, where I can specify partitioning and ordering.

Common window functions are ROW_NUMBER, RANK, DENSE_RANK, SUM, AVG, LAG, LEAD.

Example: Suppose I want to calculate running totals of sales per customer.

```
SELECT customer_id, order_id, order_date, amount,
    SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) as running_total
FROM orders;
```

Here, the SUM function calculates a cumulative total per customer, ordered by date. Each row still shows up, but I get an additional column showing the running total.

I use window functions a lot for ranking, time-series analysis, calculating moving averages, and comparing values between rows (like current vs previous).

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

18. What does ARRAY_AGG do and when would you use it?

ARRAY_AGG is an aggregation function in BigQuery that collects values from multiple rows and returns them as an array. Instead of giving me a single value like SUM or COUNT, it groups all the values together into a list.

For example, if I want to see all order IDs for each customer:

```
SELECT customer_id, ARRAY_AGG(order_id) as order_list

FROM orders

GROUP BY customer_id;
```

This will give me one row per customer, with an array of all their order IDs.

I use ARRAY_AGG when I need to consolidate many rows into one and preserve detail, like storing all product names a customer bought, or all tags attached to an article. It's very useful when working with nested or semi-structured data, because arrays are a natural way to store one-to-many relationships in BigQuery.

19. Explain nested and repeated fields in BigQuery and when to use them (semi-structured data).

BigQuery supports complex data structures like nested and repeated fields, which are helpful when dealing with semi-structured data such as JSON.

- A nested field is basically a STRUCT, where a column itself contains multiple subfields (like an object).
- A repeated field is an ARRAY, where a column contains multiple values of the same type.

Example schema:

```
{
 "customer_id": 101,
 "name": "John",
 "orders": [
  {
   "order_id": "A1",
   "amount": 200
  },
  {
   "order_id": "A2",
   "amount": 150
  }
 ]
}
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Here, orders is a repeated field (array), and each element inside it is a nested STRUCT with order_id and amount.

I use nested and repeated fields when the source data is naturally hierarchical (like JSON, logs, or event data). Instead of flattening everything into multiple tables, I can keep the structure intact in BigQuery and query it with SQL. This reduces joins and makes it more efficient to store one-to-many relationships in a single row.

20. What is the UNNEST operator and how is it used to work with arrays / repeated fields?

The UNNEST operator in BigQuery is used to flatten an array (repeated field) into individual rows, so that I can work with each element separately.

For example, if I have a table where each customer row contains an array of orders:

```
SELECT customer_id, orders

FROM customers;
```

The orders column might be an array. To query each order, I need to UNNEST it:

```
SELECT customer_id, order.order_id, order.amount

FROM customers,

UNNEST(orders) as order;
```

This will expand the array into multiple rows one row per order per customer.

Without UNNEST, the data stays inside the array and I can't easily filter or aggregate on its elements. So I use UNNEST whenever I want to flatten repeated fields and analyze them at row-level.

21. What are user-defined functions (UDFs) in BigQuery and how do you create/use them?

User-defined functions in BigQuery allow me to write my own custom functions when the built-in SQL functions are not enough. They help me reuse logic and keep my queries cleaner. I can create them in two ways: using SQL or using JavaScript.

With SQL UDFs, I define a function using SQL expressions. For example, if I want to standardize names by trimming spaces and making the first letter uppercase, I can create a temporary SQL UDF like this:

```
CREATE TEMP FUNCTION normalize_name(name STRING) AS (

  INITCAP(TRIM(name))

);

SELECT normalize_name("   john doe   ");
```

This returns "John Doe".

With JavaScript UDFs, I can write more complex logic, like advanced string manipulation or regex matching that SQL doesn't handle easily.

In real projects, I use UDFs when I need consistent custom calculations across multiple queries, like formatting phone numbers, applying business rules, or handling special data transformations.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

22. What are PIVOT and UNPIVOT operators and when would you use them?

PIVOT and UNPIVOT are used to reshape data. They help me rotate rows into columns or columns into rows, depending on the analysis I want.

- PIVOT takes unique values from one column and turns them into multiple columns. For example, if I have monthly sales data, I can pivot the table so each month becomes a column showing sales totals.

```
SELECT *

FROM sales

PIVOT(SUM(amount) FOR month IN ('Jan', 'Feb', 'Mar'));
```

This gives me one row per product with Jan, Feb, and Mar as separate columns.

- UNPIVOT does the opposite. It takes multiple columns and turns them into rows. For example, if I have columns like Jan_sales, Feb_sales, and Mar_sales, I can unpivot them so that the result has a "month" column and a "sales" column.

```
SELECT *

FROM sales

UNPIVOT(sales_amount FOR month IN (Jan_sales, Feb_sales, Mar_sales));
```

I use PIVOT when I want a wide report format, like a pivot table in Excel, and UNPIVOT when I want to normalize wide data into a long format that's easier for analysis, aggregations, or joins.

23. What is BigQuery ML (BQML) and what can you build with it?

BigQuery ML is a feature that lets me build and use machine learning models directly in BigQuery with SQL. The big advantage is that I don't need to export data to another ML platform I can train, evaluate, and make predictions right where the data lives.

With BQML I can build different types of models:

- Regression models like linear and logistic regression for predicting numbers or binary outcomes.
- Classification models for predicting categories.
- Clustering models using k-means.
- Time series forecasting models using ARIMA or Prophet+.
- Recommendation models for things like product recommendations.
- I can also import/export TensorFlow and XGBoost models to use them in BigQuery.

For example, I can build a churn prediction model with just SQL:

```
CREATE OR REPLACE MODEL my_dataset.churn_model

OPTIONS(model_type='logistic_reg') AS

SELECT features, label

FROM training_data;
```

I use BQML when I want quick ML solutions without moving data out of BigQuery, and it's very useful for analytics teams that are strong in SQL but not in Python or ML frameworks.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. How does BigQuery handle security (IAM, roles, encryption, VPC Service Controls, audit logging)?

BigQuery provides multiple layers of security to make sure data is protected and access is controlled properly.

- IAM roles: I can assign permissions at the project, dataset, table, or even column level. For example, some users may only have read access, while others can run queries or manage datasets.
- Column-level and row-level security: I can restrict access to sensitive data so that people only see the columns or rows they are allowed to. For example, only HR can see salary details.
- Encryption: All data is automatically encrypted at rest and in transit. If needed, I can use customer-managed keys for extra control.
- VPC Service Controls: These provide an additional boundary around BigQuery resources to prevent data leakage outside the organization.
- Audit logging: Every query and access is logged in Cloud Audit Logs. This helps me monitor usage, detect unusual behavior, and prove compliance.

Together, these features make BigQuery secure by default, but also flexible so I can enforce the right level of security depending on business and compliance needs.

25. What is the difference between legacy SQL and standard SQL in BigQuery?

BigQuery originally had its own SQL dialect called Legacy SQL, but now the default is Standard SQL, which follows the ANSI SQL 2011 standard.

Legacy SQL had different syntax and some unique functions, but it wasn't very consistent with other databases, which made it harder for people to migrate queries. Standard SQL is much more familiar for anyone who already knows SQL, supports advanced features like window functions, arrays, and structs, and allows better interoperability.

For example, in Legacy SQL you used a special syntax like SELECT TOP(10) ... while in Standard SQL you use the more common SELECT ... LIMIT 10.

Today, Standard SQL is recommended for everything because it's more powerful, portable, and supports features like JOINs on expressions, subqueries in SELECT, and better error handling. Legacy SQL is still available for backward compatibility, but I always prefer Standard SQL in new projects.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

26. What are materialized views in BigQuery and when should you use them?

Materialized views are precomputed views in BigQuery that store the results of a query physically, unlike normal views which just store the query definition. This means materialized views can make queries much faster and cheaper because the heavy computation is done once and automatically kept up to date by BigQuery as the base table changes.

For example, if I have a huge fact table with billions of rows and I often run queries like daily sales totals, I can create a materialized view that stores those aggregates. Then my queries on top of it only read the precomputed results instead of scanning the entire base table.

I use materialized views when:

- There are frequent repeated queries on the same aggregations.
- I want to save costs by scanning less data.
- I need faster performance for dashboards or BI tools.

They are best for queries with predictable patterns like aggregations and filters, not for highly dynamic ad-hoc queries.

27. How do you manage data lifecycle in BigQuery (table/partition expiration, backups, retention policies)?

Managing data lifecycle in BigQuery is important to control costs and ensure compliance. There are several ways to do this:

- Table expiration: I can set an expiration time for a whole table, so it automatically gets deleted after a certain number of days. This is useful for temporary or staging tables.
- Partition expiration: For partitioned tables, I can set expiration times at the partition level. For example, keep only the last 90 days of log data and automatically drop older partitions. This avoids manual cleanup.
- Dataset default expiration: I can set a default expiration for all tables in a dataset so that every new table created inherits it.
- Backups and recovery: BigQuery supports table snapshots and time travel. With time travel, I can query a table as it existed up to 7 days in the past. Table snapshots allow me to take a point-in-time copy of a table for longer-term backup.
- Retention policies: By combining expiration settings, snapshots, and access control, I can enforce data retention that meets business or regulatory requirements.

In practice, I usually configure partition expiration for log or event tables, use snapshots for important backups, and rely on time travel for quick accidental deletion recovery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

28. What is a federated query and when should you use it?

A federated query in BigQuery means running SQL queries on data that lives outside of BigQuery, without first loading it into BigQuery storage. BigQuery can directly query external sources like Google Cloud Storage (CSV, Parquet, Avro, JSON), Cloud Bigtable, Google Drive (Google Sheets), and even Cloud SQL.

For example, I could query a CSV file sitting in Cloud Storage like it's a table:

```
SELECT *

FROM EXTERNAL_QUERY("my_project.my_dataset.my_external_table")
```

The advantage is I don't need to copy or duplicate data into BigQuery. It's very useful for ad-hoc analysis, quick proof-of-concepts, or when the data changes frequently in the external source.

However, federated queries are not always as fast or cheap as querying native BigQuery tables, because data may need to be read across networks. I use them when I want to avoid duplication or need to combine live external data with BigQuery tables quickly.

29. What tools can you use to orchestrate BigQuery data pipelines (Dataflow, Composer/Airflow, Cloud Functions, etc.)?

There are several tools I can use to orchestrate and manage data pipelines that load or transform data in BigQuery:

- Cloud Dataflow (Apache Beam): Great for building complex ETL/ELT pipelines with batch and streaming data, often used for transforming and enriching data before loading into BigQuery.
- Cloud Composer (managed Airflow): Best for orchestrating workflows across multiple services. For example, I can schedule jobs to pull data from APIs, process it with Dataflow, and load it into BigQuery.
- Cloud Functions / Cloud Run: Useful for event-driven pipelines. For example, when a file lands in Cloud Storage, a Cloud Function can trigger a load into BigQuery.
- BigQuery Scheduled Queries: For lightweight orchestration, I can simply schedule recurring SQL transformations inside BigQuery itself.
- Third-party tools: Tools like dbt, Fivetran, and Informatica are also commonly used with BigQuery.

In real projects, I often combine these. For example, raw data lands in GCS, Dataflow transforms it, Composer schedules the workflows, and BigQuery scheduled queries handle recurring aggregations.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. How do you monitor and debug BigQuery queries (job history, INFORMATION_SCHEMA, query plan, Cloud Monitoring)?

To monitor and debug BigQuery queries, I use several built-in tools:

- Job History: In the BigQuery console, I can check query history for each job. It shows execution time, bytes processed, and status.
- INFORMATION_SCHEMA tables: BigQuery exposes metadata tables like INFORMATION_SCHEMA.JOBS_BY_USER or INFORMATION_SCHEMA.TABLES, where I can run queries to see which jobs are running, how much data is scanned, and costs. This helps track usage across projects.
- Query Plan / Execution Details: After running a query, I can open the execution details to see a visual query plan. It shows each stage of execution, how much data was read, shuffled, or processed. This helps me identify performance bottlenecks.
- Cloud Monitoring (Stackdriver): I can set up dashboards and alerts for query performance, slot utilization, and errors. For example, I can alert if queries are consuming too many slots or if costs spike suddenly.
- Audit Logs: Cloud Audit Logs track who ran what query and when, which helps for debugging both security and operational issues.

In practice, when debugging a slow or costly query, I first look at the execution details, then check INFORMATION_SCHEMA for cost/slot usage, and finally use Cloud Monitoring for long-term patterns.

31. What are the main advantages of using BigQuery?

The main advantages of BigQuery are scalability, simplicity, and performance.

- It's fully managed and serverless, so I don't need to manage infrastructure, clusters, or indexes. I just write SQL and BigQuery takes care of the execution.
- It's extremely scalable. I can query terabytes or even petabytes of data within seconds because BigQuery runs queries in parallel across thousands of slots.
- It stores data in columnar format and uses advanced compression, which makes analytical queries very fast and cost-efficient.
- It integrates very well with other GCP services like Dataflow, Pub/Sub, Cloud Storage, Looker Studio, and also supports machine learning with BQML.
- Cost flexibility is another advantage. I can use on-demand pricing and pay only for scanned data, or flat-rate slot reservations if I need predictable pricing.
- It also has strong features for governance and security, like IAM roles, row-level and column-level access, and encryption by default.

Overall, BigQuery makes it much easier to do large-scale analytics with minimal operational overhead.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

32. How does BigQuery handle very large datasets (columnar storage, compression, automatic sharding/parallelism)?

BigQuery is designed to handle huge datasets efficiently because of how its architecture works:

- It uses columnar storage in Colossus, which means data is stored by columns instead of rows. This makes it very efficient for analytical queries that often need only a few columns.
- Data is automatically compressed with advanced techniques, reducing storage costs and the amount of data read during queries.
- It supports automatic sharding and parallelism. The Dremel query engine breaks queries into smaller tasks and runs them in parallel across thousands of slots, scanning only the needed partitions and columns.
- There's no need for me to manually create indexes or pre-shard the data. BigQuery handles distribution automatically in the backend.
- Features like partitioning and clustering also help reduce the amount of data scanned, which speeds up queries on large tables.

Because of these design choices, BigQuery can process datasets at the petabyte scale while still delivering results in seconds.

33. How do you export data from BigQuery (to GCS in CSV/JSON/Avro/Parquet, etc.)?

Exporting data from BigQuery is usually done to Google Cloud Storage. I can export whole tables or the results of queries into different file formats.

- The supported export formats are CSV, JSON, Avro, Parquet, and ORC.
- To export a table, I can use the BigQuery console, the bq command-line tool, or the API. For example, with the CLI:

```
bq extract --destination_format=PARQUET my_dataset.my_table
gs://my_bucket/my_table.parquet
```

- For large tables, BigQuery automatically splits the export into multiple files in parallel, which makes it more scalable.
- I can also use scheduled exports or Dataflow/Composer pipelines if I need recurring exports.

In practice, I often export in Parquet or Avro format because they are columnar and efficient for downstream analytics, while CSV or JSON are better when the data is meant for simple sharing or interoperability with non-analytical systems.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

34. What is the BigQuery sandbox and what are its limits?

The BigQuery sandbox is a free environment that lets me use BigQuery without setting up billing. It's useful for learning, experimenting, or small projects where I don't want to spend money.

The sandbox has some limits:

- Storage limit is 10 GB per project.
- Query processing limit is 1 TB per month.
- Tables created in the sandbox expire after 60 days.
- Some enterprise features like streaming inserts, Data Transfer Service, and table snapshots are not available.

The good thing is I can still use most SQL features, datasets, and queries. When I hit the limits or need production features, I can easily enable billing and move out of the sandbox.

35. How does BigQuery ensure data consistency (read-after-write semantics, eventual consistency caveats)?

BigQuery provides strong consistency for queries. That means once I load or insert data into a table, I can immediately query it and see the changes this is read-after-write consistency.

However, there are a couple of caveats:

- Streaming inserts are not always instantly visible. Usually, they become available within a few seconds, but in rare cases it can take longer.
- Metadata changes, like creating or updating datasets or tables, may take a few seconds to propagate across regions.

In practice, for batch loads and query results, I can rely on strong consistency. For streaming, I design my systems to tolerate a small delay before data appears in queries.

36. What is the difference between append and overwrite jobs when loading data into BigQuery?

When I load data into BigQuery, I can choose the write disposition:

- Append (WRITE_APPEND): New data is added to the existing table without removing the old data. I use this for incremental loads, like adding today's logs into a history table.
- Overwrite (WRITE_TRUNCATE): The existing table is replaced with the new data. I use this when I want to completely refresh a table with the latest dataset, for example, when recreating a daily snapshot.
- There is also WRITE_EMPTY, which fails the job if the table is not empty useful for safety checks.

Choosing append or overwrite depends on the use case. Append is good for continuous data growth, overwrite is good for rebuilding tables.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

37. How do you implement data governance in BigQuery (IAM, labels, data catalog, policies)?

Data governance in BigQuery involves controlling access, organizing datasets, and tracking metadata so that data is secure, discoverable, and compliant.

- IAM roles: I can control access at the project, dataset, table, column, and even row level. For example, finance users may only see finance tables, and PII columns can be restricted with column-level security.
- Labels: I can tag datasets and tables with labels like environment=prod or department=marketing. This helps in organizing data and also tracking costs by label in billing reports.
- Data Catalog: This is Google's metadata management tool. I can use it to document datasets, apply business metadata, classify sensitive data, and let users search for data assets.
- Policies and retention: I can set table and partition expiration policies to enforce data lifecycle rules. I can also use policy tags for data classification (like marking fields as confidential).
- Audit logs: BigQuery integrates with Cloud Audit Logs so I can monitor who accessed what data and when.

By combining IAM, labels, Data Catalog, and policies, I can create a robust governance framework where access is least-privilege, metadata is well-documented, and compliance requirements are met.

38. How do you ensure GDPR / privacy compliance when storing data in BigQuery?

To ensure GDPR and privacy compliance in BigQuery, I follow a mix of design, security, and governance practices.

- First, I minimize storing personal data unless it's absolutely required. If I must store PII, I use techniques like pseudonymization or anonymization.
- I use column-level security to restrict access to sensitive fields like email or phone numbers. Only authorized users can query those columns.
- Row-level security helps me make sure that users only see the rows they're allowed to see, for example filtering data by region or department.
- BigQuery encrypts all data by default, and I can use customer-managed encryption keys for stricter control.
- Data Catalog and policy tags allow me to classify fields as sensitive and enforce consistent governance.
- I also make use of audit logs to track who accessed what data, which is important for GDPR reporting.
- Finally, I apply data retention policies so data doesn't live longer than needed, aligning with the "right to be forgotten."

So GDPR compliance in BigQuery is mainly about limiting access, controlling retention, encrypting, and auditing usage.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

39. When should you use BigQuery instead of transactional databases (MySQL) or document stores (MongoDB)?

I use BigQuery when I need large-scale analytics, not when I need fast transactions.

- BigQuery is ideal when the data is very large (terabytes or petabytes), and I need to run analytical queries like aggregations, trend analysis, and reporting. It's great for dashboards, BI, data science, and machine learning.
- MySQL or other transactional databases are better for OLTP use cases where there are frequent inserts, updates, and deletes, and where I need low-latency row-level reads (for example, banking systems or e-commerce apps).
- MongoDB or other document stores are good for flexible schemas and operational workloads with unstructured or semi-structured data, like user profiles or application logs.

So the rule of thumb: if the workload is analytical and read-heavy on massive datasets, I use BigQuery. If it's transactional with frequent updates and small data volumes, I stick with MySQL or MongoDB.

40. Can you publicly share datasets, tables, or queries in BigQuery? If so, how?

Yes, BigQuery lets me share datasets or tables publicly by adjusting access permissions.

- At the dataset or table level, I can grant roles like bigquery.dataViewer to "allAuthenticatedUsers" or "allUsers." This makes the data accessible publicly, similar to how public datasets are shared.
- I can also share a dataset with specific users, groups, or domains by granting them access via IAM.
- For queries, I can share saved queries in the BigQuery console or export results to Google Sheets or Looker Studio and then share those with others.
- Another option is authorized views: I can create a view that exposes only specific columns or rows, and share the view without exposing the raw table.

This is commonly used when publishing open datasets for the community, while still applying security best practices.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

41. How do you create and manage views in BigQuery?

In BigQuery, a view is basically a saved SQL query that acts like a virtual table. It doesn't store data itself; instead, every time I query a view, BigQuery runs the underlying SQL.

To create a view, I can use the console, CLI, or SQL. For example:

```
CREATE OR REPLACE VIEW my_dataset.top_customers AS

SELECT customer_id, SUM(amount) as total_spent

FROM orders

GROUP BY customer_id

ORDER BY total_spent DESC;
```

Now, my_dataset.top_customers is a view that I can query like a table.

For management:

- Views automatically reflect updates to the underlying tables.
- I can grant access to views separately from the underlying tables, which is good for governance (for example, giving analysts access only to an aggregated view, not raw PII data).
- I can replace or update views easily with CREATE OR REPLACE VIEW.
- For performance-sensitive cases, I sometimes use materialized views instead, since they store results and refresh automatically.

Views are a good way to enforce consistent business logic and simplify query writing for end users.

42. What are common BigQuery SQL errors and how do you resolve them?

In my experience, the most common errors in BigQuery come from syntax issues, mismatched column names, or resource limits. For example, sometimes if I forget to use backticks around reserved keywords or dataset.table names, the query fails. Another one I see often is duplicate column names when joining tables BigQuery needs explicit aliases.

On the performance side, queries can fail if they scan too much data or exceed slot limits. In those cases, I usually fix the query by adding filters, partitioning, or clustering so less data is scanned.

Streaming inserts can also cause errors if schema mismatches happen I usually make sure the schema matches exactly, or I use schema evolution features.

So in practice, I debug by looking at the error message first (they're usually very descriptive), then I check the query plan or INFORMATION_SCHEMA jobs to see where it's failing.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

43. What are the various ways to interact with BigQuery (Console, bq CLI, REST API, client libraries, connectors)?

I've used BigQuery in different ways depending on the situation. For quick analysis or exploring datasets, I prefer the web console because it's visual and easy. When I want to automate things like data loads or exports, I use the bq CLI in scripts.

For programmatic access, I've worked with the Python client library that's very common when integrating BigQuery into pipelines. Under the hood, everything goes through the REST API.

On the orchestration side, I often use connectors. For example, Dataflow has a BigQuery connector, so I can write directly to BigQuery. BI tools like Looker, Tableau, and Power BI also connect natively.

So, depending on the task quick query, automation, coding, or visualization I choose the right interface.

44. How do you use BigQuery for data visualization and reporting, and what BI tools commonly connect to it?

In most projects, BigQuery is the central warehouse and dashboards sit on top of it. I usually expose the processed data through views or materialized views, so analysts and BI tools don't have to deal with raw tables.

The most common tool I've used is Looker Studio (formerly Data Studio), since it's free and integrates very well. In enterprise setups, I've seen a lot of Tableau and Power BI as well, both of which have BigQuery connectors. Looker itself also works seamlessly with BigQuery.

To improve dashboard performance, I sometimes use BI Engine, which caches queries in memory. That helps when dashboards are hit by a lot of users.

So, the process is: clean and transform in BigQuery, create views for reporting, and then connect the BI tool of choice for visualization.

45. How do you design and implement data pipelines that target BigQuery (ETL vs ELT, streaming vs batch, dedupe strategy)?

The way I design a pipeline depends on the type of data and how fast it needs to be available.

If freshness isn't critical, I usually prefer batch loads. Data lands in Cloud Storage first, and then I load it into BigQuery in formats like Parquet or Avro. It's cost-effective and works well for daily or hourly pipelines.

If near real-time analytics is needed, I go with streaming pipelines usually Pub/Sub + Dataflow writing into BigQuery. That way, data is queryable within seconds.

On the transformation side, I've used both ETL and ELT. With ETL, I transform data before loading (for example, in Dataflow). With ELT, I load raw data first and then transform using SQL inside BigQuery. Since BigQuery is powerful at SQL transformations, I find ELT works really well for many cases.

For deduplication, I usually rely on unique IDs and then use BigQuery SQL for example, ROW_NUMBER() with QUALIFY to keep only the latest record. Another option is using MERGE for upserts.

So overall, I choose between batch/streaming and ETL/ELT based on latency needs, transformation complexity, and cost.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

46. How can you remove duplicate rows from a BigQuery table while preserving the original table name?

If I want to remove duplicates but keep the same table name, I usually use a ROW_NUMBER() or RANK() window function to identify duplicates and then rewrite the table.

For example, let's say the duplicates are based on customer_id and order_id. I would run something like:

```
CREATE OR REPLACE TABLE my_dataset.my_table AS

SELECT *

FROM (

  SELECT t.*,

       ROW_NUMBER() OVER(PARTITION BY customer_id, order_id ORDER BY update_time
DESC) AS rn

  FROM my_dataset.my_table t

)

WHERE rn = 1;
```

This keeps only one row per unique key, in this case the latest by update_time. The CREATE OR REPLACE TABLE overwrites the old table with the deduplicated results, so the table name stays the same.

This approach is common when I want to clean data but not create a new table just for that purpose.

47. How do you resolve the error: "UPDATE or DELETE DML statements are not supported over table ... with streaming buffer"?

This happens when I try to update or delete rows in a table that still has recent streaming inserts. BigQuery puts streaming data into a buffer before it's fully written to storage, and DML statements (like UPDATE or DELETE) don't work while data is in that buffer.

The way I usually resolve it is either:

- Wait a few minutes until the streaming buffer is flushed into permanent storage, and then run the DML.
- If I can't wait, I can work around it by writing a new table. For example, I can use a query with the desired UPDATE/DELETE logic and then CREATE OR REPLACE the table.

In real projects, I try to avoid running DML directly on tables with heavy streaming loads. Instead, I design pipelines to stage the data, clean it, and then merge it into the main table in batch mode.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

48. How do you find the Google Cloud Storage URI (gs://...) for an object using the Cloud Console?

In the Cloud Console, I go to the **Cloud Storage browser**, then navigate to the bucket and folder where my object is stored. Once I click on the object, the details panel opens. In there, I can see the **"gs://" URI** listed, something like gs://my-bucket/my-folder/my-file.csv.

Another way is to click the three-dot menu next to the object and choose **Copy URL**, which gives me the gs:// path directly.

I usually use this URI when exporting from BigQuery, configuring Dataflow, or sharing file locations with others.

Section B SQL

1. Write a SQL query to select the top 10 highest salaries from an employee table.

```
SELECT employee_id, employee_name, salary

FROM employee

ORDER BY salary DESC

LIMIT 10;
```

Here I'm ordering by salary in descending order so the highest salaries come first, and then using LIMIT 10 to get the top 10.

2. Write a SQL query to select the top 10 most expensive products.

```
SELECT product_id, product_name, price

FROM products

ORDER BY price DESC

LIMIT 10;
```

This is similar logic order by price descending and limit the result to 10. That way I get the most expensive products.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

3. Write a SQL query to calculate the average sales per month from a sales table.

Assuming the sales table has a sale_date and amount column:

```
SELECT
  EXTRACT(YEAR FROM sale_date) AS year,
  EXTRACT(MONTH FROM sale_date) AS month,
  AVG(amount) AS avg_sales
FROM sales
GROUP BY year, month
ORDER BY year, month;
```

Here I'm extracting year and month from the sale_date, grouping by them, and then calculating the average sales amount per month. Ordering makes it easy to see the trend over time.

4. Write a SQL query to find the total number of orders placed by each customer.

```
SELECT customer_id, COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_id
ORDER BY total_orders DESC;
```

Here I'm grouping by customer_id and counting orders. Adding ORDER BY helps show who placed the most orders, which is often useful in analysis.

5. Write a SQL query to find the total number of deliveries for each product.

```
SELECT product_id, COUNT(delivery_id) AS total_deliveries
FROM deliveries
GROUP BY product_id
ORDER BY total_deliveries DESC;
```

I'm assuming there's a deliveries table with product_id and delivery_id. The logic is similar: group by product, count deliveries, and order by the count.

6. Write a SQL query to retrieve all records from a table where the date is within the last 30 days.

```
SELECT *
FROM sales
WHERE sale_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

This gives me all rows where the sale_date is in the last 30 days. CURRENT_DATE ensures it's dynamic, so it works whenever the query is run.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. How do you handle NULL values in BigQuery SQL queries (functions and patterns)?

In an interview I'd say: NULLs mean "unknown/missing" and you should decide the business rule first do you ignore them, replace them with a default, or treat them as a special case? Technically I rely on a few standard patterns and functions:

- Use IS NULL / IS NOT NULL to explicitly filter or branch on NULLs.
- Use COALESCE(expr1, expr2, ...) to provide a default value (returns the first non-NULL). Example: COALESCE(discount, 0) to treat missing discount as 0.
- Use IFNULL(expr, default) when you only need two arguments (shorthand for COALESCE).
- Use NULLIF(a, b) to turn a value into NULL (useful to avoid divide-by-zero: value / NULLIF(divisor, 0)).
- Use CASE WHEN ... THEN ... ELSE ... END for complex NULL-aware logic.
- Use SAFE_CAST() to prevent errors when casting malformed strings it returns NULL instead of failing.
- Remember COUNT(column) ignores NULLs while COUNT(*) counts rows; use the one that matches your intent.
- When joining, be careful with NULL join keys decide whether to pre-filter WHERE key IS NOT NULL or handle the NULLs explicitly.
- For arrays/structs, use ARRAY_LENGTH or SAFE_OFFSET etc., and UNNEST() only after checking for NULL/empty arrays if needed.

Example patterns:

- Fill defaults: SELECT COALESCE(email, 'unknown@example.com') AS email_clean FROM users;
- Avoid crash: SELECT amount / NULLIF(divisor, 0) FROM table;
- Count non-null values: SELECT COUNT(last_login) FROM users; (counts only rows with a non-NULL last_login)

I always pick the pattern that matches the business meaning of the missing value and make it explicit in the SQL so others won't be surprised.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

8. Write a SQL query to count the number of unique users who logged in during the last week.

I'd clarify in an interview that "last week" can mean last 7 days or the previous calendar week; here I'll show last 7 days (including today). If you want calendar-week semantics I'd use DATE_TRUNC with WEEK.

Exact distinct count (last 7 days):

```
SELECT COUNT(DISTINCT user_id) AS unique_users_last_7_days

FROM `project.dataset.logins`

WHERE login_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY);
```

If the table is very large and an approximate answer is acceptable (much cheaper / faster):

```
SELECT APPROX_COUNT_DISTINCT(user_id) AS approx_unique_users_last_7_days

FROM `project.dataset.logins`

WHERE login_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY);
```

If you want previous calendar week (Mon–Sun), example:

```
SELECT COUNT(DISTINCT user_id) AS unique_users_prev_calendar_week

FROM `project.dataset.logins`

WHERE DATE_TRUNC(login_date, WEEK(MONDAY)) =
DATE_SUB(DATE_TRUNC(CURRENT_DATE(), WEEK(MONDAY)), INTERVAL 1 WEEK);
```

I'd mention which interpretation I used when answering so the interviewer knows I considered ambiguity.

9. Write a SQL query to find the maximum and minimum order values from an orders table.

Simple and direct MAX and MIN ignore NULLs by default:

```
SELECT

  MAX(order_amount) AS max_order_value,

  MIN(order_amount) AS min_order_value

FROM `project.dataset.orders`;
```

If you want to ignore cancelled orders or consider only completed orders:

```
SELECT

  MAX(order_amount) AS max_order_value,

  MIN(order_amount) AS min_order_value

FROM `project.dataset.orders`

WHERE status = 'COMPLETED';
```

If you need the full row(s) with the max or min value (not just the number), you can use QUALIFY or window functions:

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
-- Row(s) with max order
SELECT *
FROM `project.dataset.orders`
QUALIFY order_amount = MAX(order_amount) OVER ();
```

10. Write a SQL query to group sales data by product category and calculate total sales for each category.

If sales already has product_category:

```
SELECT
  product_category,
  SUM(amount) AS total_sales,
  COUNT(*) AS transaction_count
FROM `project.dataset.sales`
GROUP BY product_category
ORDER BY total_sales DESC;
```

If category is in a separate products table (join example):

```
SELECT
  p.category_name,
  SUM(s.amount) AS total_sales,
  COUNT(*) AS transaction_count
FROM `project.dataset.sales` s
JOIN `project.dataset.products` p
  ON s.product_id = p.product_id
GROUP BY p.category_name
ORDER BY total_sales DESC;
```

I usually include ORDER BY total_sales DESC so stakeholders immediately see the top categories, and I add COUNT(*) to show volume as well as dollars.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. Write a SQL query to find the percentage of total sales contributed by each product.

I'd answer this in an interview like: compute each product's sales and divide by the grand total use a window function or a cross join to the total. Here's a concise BigQuery-friendly way using a window:

```
SELECT
  product_id,
  SUM(amount) AS product_sales,
  ROUND(100 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS pct_of_total_sales
FROM `project.dataset.sales`
GROUP BY product_id
ORDER BY product_sales DESC;
```

Explanation: SUM(SUM(amount)) OVER () computes the grand total across all products (the double SUM: aggregate then window). Then we divide to get the share and multiply by 100. I use ROUND(..., 2) to show percent with two decimal places.

12. Write a SQL query to retrieve the last 5 transactions for each customer.

In an interview I'd say: use a window function to rank transactions per customer by date and keep the top 5. In BigQuery you can use QUALIFY to filter after windowing which keeps the query tidy:

```
SELECT
  customer_id,
  transaction_id,
  transaction_date,
  amount
FROM `project.dataset.transactions`
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY transaction_date
DESC, transaction_id DESC) <= 5
ORDER BY customer_id, transaction_date DESC;
```

Explanation: ROW_NUMBER() partitions by customer_id and orders by newest transactions first. QUALIFY filters rows by that row number, returning up to 5 most recent rows per customer. Adding transaction_id to ORDER BY provides deterministic tie-breaking.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

13. Write a SQL query to create a temporary table and insert data into it.

I'd describe two common approaches: create-from-select (most common for staging) or explicit CREATE + INSERT. Example 1 (create temp table from query):

```
-- Create a temp table from a query result

CREATE TEMP TABLE temp_top_products AS

SELECT product_id, SUM(amount) AS total_sales

FROM `project.dataset.sales`

GROUP BY product_id

HAVING SUM(amount) > 10000;
```

Example 2 (create schema and insert rows):

```
-- Define schema then insert rows

CREATE TEMP TABLE temp_lookup (

  code STRING,

  description STRING

);


INSERT INTO temp_lookup (code, description)

VALUES

  ('A', 'Alpha'),

  ('B', 'Beta');
```

Explanation: CREATE TEMP TABLE creates a session-scoped table (auto-dropped at session end). Use the AS SELECT form to materialize query results quickly, or define columns explicitly and INSERT when you have small lookup data or need to build the table step-by-step.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. Write a SQL query that uses a window function to rank products based on sales within each category.

```
SELECT
  product_id,
  category,
  total_sales,
  RANK() OVER (PARTITION BY category ORDER BY total_sales DESC) AS category_rank
FROM (
  SELECT
    product_id,
    category,
    SUM(amount) AS total_sales
  FROM `project.dataset.sales`
  GROUP BY product_id, category
) t
ORDER BY category, category_rank;
```

Explanation: I first aggregate sales per product within each category, then use RANK() with PARTITION BY category to assign ranks within each category. RANK() gives ties the same rank and will skip ranks after ties; use DENSE_RANK() if you don't want gaps.

15. Write a SQL query to rank products based on total sales (global rank).

```
SELECT
  product_id,
  total_sales,
  RANK() OVER (ORDER BY total_sales DESC) AS global_rank
FROM (
  SELECT
    product_id,
    SUM(amount) AS total_sales
  FROM `project.dataset.sales`
  GROUP BY product_id
) t
ORDER BY global_rank;
```

Explanation: Aggregate total sales per product, then use RANK() (or DENSE_RANK() / ROW_NUMBER() depending on tie behavior you prefer) over the full result to compute a global ranking across all products.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

16. Write a SQL query to count the number of unique products sold in the last week.

```
SELECT
  COUNT(DISTINCT product_id) AS unique_products_last_7_days
FROM `project.dataset.sales`
WHERE sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY);
```

Explanation: This counts distinct product_ids with sale_date in the last 7 days (including today). If you need the previous calendar week instead, use DATE_TRUNC(..., WEEK) logic I'd clarify which interpretation the interviewer wants.

17. Write a SQL query to group sales data by product and calculate total revenue for each product.

```
SELECT
  p.product_id,
  p.product_name,
  SUM(s.amount) AS total_revenue,
  COUNT(*) AS transactions
FROM `project.dataset.sales` s
LEFT JOIN `project.dataset.products` p
  ON s.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_revenue DESC;
```

Explanation: I aggregate SUM(amount) per product to get revenue, join to the products table to show friendly names, and include COUNT(*) to show volume. Ordering by total_revenue DESC surfaces top-earning products first.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

18. Write a SQL query to calculate the average delivery time for each product.

I'd first decide the units I want (hours, minutes, seconds). I also make sure to ignore rows where the delivered timestamp is missing or invalid. Here's a simple BigQuery example that calculates average delivery time in hours per product:

```
SELECT
  product_id,
  COUNT(1) AS deliveries_count,
  ROUND(AVG(TIMESTAMP_DIFF(delivered_at, shipped_at, HOUR)), 2) AS avg_delivery_hours
FROM `project.dataset.deliveries`
WHERE shipped_at IS NOT NULL
  AND delivered_at IS NOT NULL
  AND delivered_at >= shipped_at
GROUP BY product_id
ORDER BY avg_delivery_hours;
```

Explanation: TIMESTAMP_DIFF(..., HOUR) computes delivery time in hours per row; AVG(...) aggregates that per product. I add ROUND(...,2) for readable output and filters to avoid negative or NULL durations. If I want more precision, I'd use SECOND or compute a formatted interval.

19. How can you include an additional field (a serial suffix or row number) in a query to identify duplicate IDs?

I normally use a window function to create a sequential counter per duplicate group and either output that counter or append it to the ID as a suffix. Example: add a dup_rank and a serial_id:

```
SELECT
  id,
  ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC, created_at DESC) AS dup_rank,
  CONCAT(CAST(id AS STRING), '_', CAST(ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) AS STRING)) AS serial_id,
  *
FROM `project.dataset.my_table`;
```

Explanation: ROW_NUMBER() OVER (PARTITION BY id ORDER BY ...) gives 1,2,3... for each id. dup_rank = 1 is typically the "keep" row. serial_id concatenates the original id with the row number so every duplicate row becomes unique and traceable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

20. What BigQuery SQL would you use to retrieve each user between two dates?

I'd clarify whether you want users created between two dates, or users who had activity between two dates. I'll give both short examples.

a) Users created between two dates (on a users table with created_date as DATE):

```
SELECT *

FROM `project.dataset.users`

WHERE created_date BETWEEN DATE('2025-08-01') AND DATE('2025-08-31');
```

b) Distinct users who had activity (e.g., logins) between two dates (from a logins table):

```
SELECT DISTINCT user_id

FROM `project.dataset.logins`

WHERE login_date BETWEEN DATE('2025-08-01') AND DATE('2025-08-31');
```

If you want the full user profile for those active users, join:

```
SELECT u.*

FROM `project.dataset.users` u

JOIN (

  SELECT DISTINCT user_id

  FROM `project.dataset.logins`

  WHERE login_date BETWEEN DATE('2025-08-01') AND DATE('2025-08-31')

) a

ON u.user_id = a.user_id;
```

Explanation: BETWEEN is inclusive; I use DISTINCT when I want unique users, or joins to get full profiles.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. How can you remove duplicate records from a column in a large BigQuery table while preserving the original table name?

If I need to deduplicate rows based on one or more columns and keep the same table name, my usual safe pattern is to CREATE OR REPLACE TABLE with a query that keeps one row per key (often the latest by timestamp). Example: dedupe by dup_col, keeping the most recent row by updated_at:

```
CREATE OR REPLACE TABLE `project.dataset.my_table` AS

SELECT * EXCEPT(rn)

FROM (

  SELECT t.*,

      ROW_NUMBER() OVER (PARTITION BY dup_col ORDER BY updated_at DESC,
some_tiebreaker DESC) AS rn

  FROM `project.dataset.my_table` t

)

WHERE rn = 1;
```

Explanation & operational notes:

- CREATE OR REPLACE TABLE atomically replaces the table with the deduped result so the original table name stays unchanged.
- ROW_NUMBER() partitions by the duplicate key and orders by a timestamp (or other tiebreaker) to choose which row to keep (rn = 1).
- For very large tables this is an expensive operation (it rewrites the whole table). In production I usually:
    - Run this during a low-traffic window,
    - Or write results to a new table my_table_deduped, validate, then CREATE OR REPLACE TABLE from that smaller validated output to minimize risk,
    - Or use partition-pruning / clustering to reduce scanned bytes if applicable.
- Alternatively, for streaming-heavy tables, consider designing an upsert flow using MERGE or using unique keys at ingest time so duplicates are prevented.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar